THRIFTY – AI-Powered Personal Finance Management System

Project Report

Submitted by: [Your Name] **Department:** [Your Department] **Institution:** [Your Institution] **Date:** March 2026

> *"Thrifty helps you spend less, save more, and grow smarter — powered by Artificial Intelligence."*

Table of Contents

TOPIC 1: INTRODUCTION & PROBLEM STATEMENT

1.1 Background and Motivation

Introduction

Personal finance management is one of the most critical life skills, yet it remains one of the most neglected areas for a large percentage of the population. Millions of individuals across the globe struggle to keep track of their daily expenses, manage monthly budgets, and plan for future financial goals. The advent of digital banking, UPI transactions, and cashless payments has made money movement faster and easier, but paradoxically, it has also made it harder for individuals to consciously track where their money goes.

The Gap in Current Solutions

While several budgeting applications exist in the market — such as Mint, YNAB (You Need A Budget), and Walnut — these solutions either lack AI-powered advisory features, are geographically limited, do not support Indian payment systems like UPI, or come with steep subscription costs that deter regular users. The majority of Indian users, particularly young working professionals and college students, lack access to affordable, intelligent, and localized financial management tools.

The Motivation Behind Thrifty

Thrifty was conceptualized to fill this gap. The project was designed with three core motivations:

1. **Accessibility**: A free, web-based tool that anyone with a browser can access — no mobile-specific installation required.
2. **Intelligence**: AI-powered financial coaching that gives personalized advice based on the user's actual spending patterns, not generic suggestions.
3. **Simplicity**: A premium, intuitive UI built with glassmorphism design principles that makes financial tracking enjoyable rather than burdensome.

Thrifty is tailored for the Indian financial ecosystem, supporting INR-denominated budgeting, UPI payment tracking, and culturally relevant spending categories like festivals, education, and household expenses. It is designed to be the personal CFO (Chief Financial Officer) for every individual — regardless of their financial literacy level.

Why This Project Matters

According to a 2024 survey by the Reserve Bank of India and multiple fintech research bodies:

- Over **68% of urban millennials** do not follow a monthly budget.
- **54% of salaried employees** run out of money before the end of the month.
- Less than **12% of households** actively use a dedicated budgeting tool.
- Financial stress is directly linked to **reduced productivity, anxiety, and poor health outcomes**.

Thrifty directly addresses these challenges by providing a platform that is proactive, intelligent, and deeply personalized.

1.2 Problem Statement

Core Problem

The core problem that Thrifty addresses can be stated as follows:

> **"Modern individuals lack an accessible, intelligent, and localized tool to manage their personal finances, track spending patterns, set budgets, and receive proactive AI-driven financial advice — leading to financial stress, overspending, and poor savings habits."**

Breakdown of the Problem

The problem can be broken into five specific sub-problems:

Sub-Problem 1: Lack of Real-Time Expense Tracking Most people track expenses manually using spreadsheets or notebooks, which is time-consuming, error-prone, and quickly abandoned. There is no real-time system that aggregates their financial data into a single, visual dashboard.

Sub-Problem 2: No Proactive Budget Alerts Users are often unaware that they have exceeded their category budgets until it is too late — the month is already over and savings are zero. There is no intelligent system that dynamically monitors spending and alerts users before a budget breach occurs.

Sub-Problem 3: Generic Financial Advice Generic financial advice available online (e.g., "save 20% of your income") does not account for an individual's specific income level, spending habits, lifestyle, or financial goals. Personalised advisory is typically only available from human financial advisors who charge significant consultation fees.

Sub-Problem 4: Data Privacy and Security Many users are reluctant to use financial apps because their sensitive financial data is stored insecurely, shared with third parties, or vulnerable to breaches. A trustworthy, secure, multi-user architecture with proper data isolation is essential.

Sub-Problem 5: Complexity of Existing Tools Existing budgeting tools often have steep learning curves, cluttered interfaces, and overwhelming features that discourage non-technical users. There is a clear need for a beautifully designed, intuitive tool that anyone can use immediately.

1.3 Existing Solutions and Their Limitations

Survey of Existing Tools

The following table provides a comparative analysis of popular personal finance management tools and their limitations:

Feature	Mint	YNAB	Walnut	Thrifty
Free to Use	☑	✗ ($15/mo)	☑	☑
AI Financial Advice	✗	✗	✗	☑
Indian UPI Support	✗	✗	☑	☑
Real-time Budget Sync	☑	☑	✗	☑
Google Login	☑	☑	✗	☑
Glassmorphism UI	✗	✗	✗	☑
Open Source	✗	✗	✗	☑

Feature	Mint	YNAB	Walnut	Thrifty
Multi-AI Support	✗	✗	✗	☑

Key Limitations of Existing Solutions

Mint (Intuit): While Mint is comprehensive, it is US-centric, does not support INR or UPI, has been plagued with security concerns, and was eventually shut down in 2024 — leaving millions of users without a platform. It lacked any AI-based conversational advisor.

YNAB (You Need A Budget): YNAB follows a "zero-based budgeting" philosophy which, while effective, is complex for new users. It requires a monthly subscription fee of $15, making it inaccessible for students and low-income users. It has no AI advisor and is not available for Indian markets.

Walnut (India): Walnut was one of the better Indian expense trackers, but it focused primarily on SMS parsing for transaction detection. It lacked a comprehensive budget planner, had no AI advisory layer, and has seen declining support and maintenance.

PhonePe/Google Pay Insights: While these UPI platforms provide basic transaction history, they offer no budgeting, no goal-setting, no AI advisor, and no comprehensive financial reporting.

Conclusion: There is a clear market gap for an **intelligent, free, India-aware, multi-feature personal finance management system** — which is what Thrifty is designed to be.

1.4 Project Scope

What Thrifty Covers

The scope of the Thrifty project encompasses the following functional areas:

1. User Management: Complete user registration, login (email/password and Google OAuth), password recovery, and profile management. Each user's data is completely isolated from other users.

2. Transaction Tracking: Full CRUD (Create, Read, Update, Delete) functionality for financial transactions. Support for multiple categories (Food, Transport, Health, Shopping, Education, Entertainment, Salary, Freelance, Investment, and more), payment methods (Cash, Card, UPI, Net Banking), and date-based filtering.

3. Budget Planning: Category-based monthly budget setting with real-time synchronization against actual transactions. Color-coded progress bars, percentage-based alerts, and critical breach notifications.

4. AI Financial Advisor: Integration with Google Gemini 1.5 Flash and Anthropic Claude 3.5 Sonnet to provide personalized financial advice. The AI reads the user's actual financial data (total income, expenses, savings rate, top categories) and provides contextually relevant coaching.

5. Dashboard & Analytics: An interactive visual dashboard showing spending breakdown charts (Recharts), monthly trends, category comparisons, and key financial metrics (total income, expenses, net savings, savings rate).

6. Gamification: A points and badge system that rewards users for consistent financial tracking behavior, making the experience engaging and habit-forming.

What Thrifty Does NOT Cover (Out of Scope)

- Actual payment processing or money transfers
- Integration with live Indian bank APIs (due to RBI regulatory requirements)
- Real-time SMS parsing from bank notifications (planned for future)
- Tax filing assistance
- Investment portfolio management (planned for future)

1.5 Project Goals and Success Criteria

Primary Goals

The following primary goals define the success of the Thrifty project:

Goal 1 – Functional Completeness: Deliver a fully functional web application that covers all five core modules: User Management, Transaction Tracking, Budget Planning, AI Advisory, and Dashboard Analytics.

Goal 2 – Security: Implement industry-standard security practices including JWT-based authentication, row-level data isolation, HTTPS enforcement, and CORS protection.

Goal 3 – Performance: Achieve page load times under 2 seconds, API response times under 500ms, and smooth 60fps animations in the user interface.

Goal 4 – Usability: Design an interface that a first-time user can navigate without any training or documentation, with intuitive layouts and clear visual hierarchies.

Goal 5 – AI Integration: Successfully integrate at least two AI models (Gemini and Claude) and deliver personalized financial advice that is contextually relevant to the user's actual financial data.

Goal 6 – Deployment: Deploy the application to a publicly accessible cloud platform (Railway for backend, Vercel for frontend) with automated CI/CD pipelines.

Success Criteria

Criterion	Measurement	Target
User can register and login	End-to-end test	Pass ☑
Transaction CRUD works	API test suite	100%
Budget syncs in real-time	Manual testing	< 1s sync delay
AI advice is generated	Integration test	< 3s response
Dashboard renders charts	Visual test	All charts load
Data isolation verified	Multi-user test	100% isolated
Deployed and accessible	URL verification	Live ☑

TOPIC 2: SYSTEM OVERVIEW & OBJECTIVES

2.1 System Overview

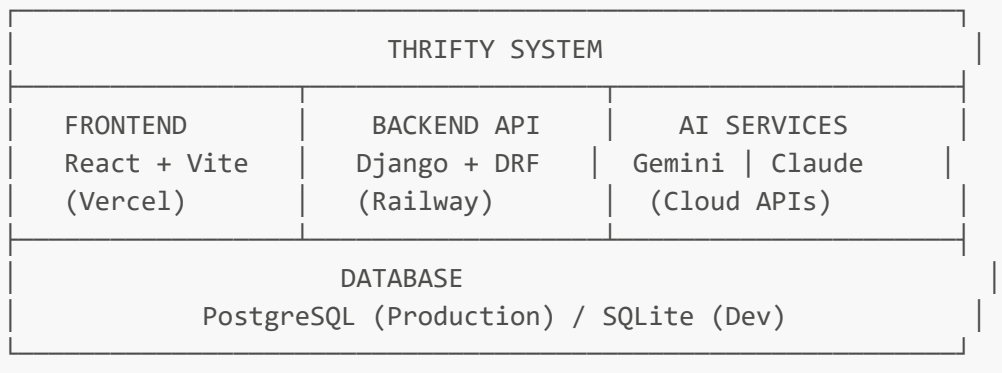
What is Thrifty?

Thrifty is a full-stack, AI-powered personal finance management web application. It provides individuals with a comprehensive platform to track their income and expenses, plan their monthly budgets, set and monitor financial goals, and receive personalized AI-driven financial coaching — all through a beautifully designed, modern web interface.

At its core, Thrifty is built on a **React + Django** architecture:

- The **Frontend** is a Single Page Application (SPA) built with **React 18** and **Vite**, featuring premium glassmorphism design, smooth Framer Motion animations, and interactive Recharts data visualizations.
- The **Backend** is a **RESTful API** built with **Django 4.x** and **Django REST Framework (DRF)**, secured with **JWT authentication** and connected to a **PostgreSQL** database in production.
- The **AI Layer** is powered by **Google Gemini 1.5 Flash** and **Anthropic Claude 3.5 Sonnet**, accessed via their respective Python SDKs from the Django backend.

High-Level Architecture



Application Flow

The typical user journey through Thrifty follows this flow:

1. **Registration/Login** → User creates an account or logs in via Google OAuth.
2. **Dashboard** → User sees their financial overview (total income, expenses, savings).
3. **Add Transactions** → User logs income/expense entries with category and amount.
4. **Set Budgets** → User defines monthly spending limits per category.
5. **Monitor Progress** → Budget progress bars update in real-time based on transactions.
6. **Consult AI Advisor** → User chats with the AI to get personalized financial advice.
7. **Review Analytics** → User explores spending charts and category breakdowns.

2.2 Key Features and Modules

Feature 1: Smart Dashboard

The dashboard is the central hub of Thrifty. It provides:

- **Summary Cards**: Total Income, Total Expenses, Net Savings, and Savings Rate — displayed as animated metric cards with color-coded indicators.
- **Spending Breakdown Chart**: A pie/donut chart showing the percentage of expenses by category using Recharts.
- **Monthly Trend Chart**: A bar chart comparing income vs. expenses across the last 6 months.
- **Recent Transactions**: A quick-view list of the 5 most recent transactions.
- **Budget Status**: A mini-view of all active budgets and their current progress.

Feature 2: Transaction Management

- Full CRUD operations for financial transactions.
- Support for 12+ expense categories and 3 income categories.
- Support for 4 payment methods: Cash, Card, UPI, Net Banking.
- Receipt upload via image field (optional).
- Advanced filtering by date range, category, and transaction type.
- Pagination for performance with large transaction histories.

Feature 3: Budget Planner

- Set monthly budget limits for any spending category.
- Real-time synchronization: Budgets instantly reflect new transactions.
- Color-coded status: Green (safe), Yellow (warning at 75%), Orange (alert at 90%), Red (critical/exceeded).
- Over-budget detection with visual and notification alerts.
- AI-generated advice triggered on budget breaches.

Feature 4: AI Financial Advisor

- Dual AI engine: Toggle between Google Gemini 1.5 Flash (fast, efficient) and Anthropic Claude 3.5 Sonnet (deep reasoning).
- Context-aware advice: The AI receives the user's real financial data before responding.
- Natural language chat interface for financial Q&A.
- Pre-built quick prompts for common financial questions.

Feature 5: Authentication & User Profiles

- Standard email/password registration with validation.
- Google OAuth 2.0 social login via Firebase Authentication.
- JWT-based session management with automatic token refresh.
- Forgot/Reset password via email link.
- User profile management (name, profile picture, preferences).

2.3 System Objectives

Primary Objectives

The system is designed to meet the following primary objectives:

Objective 1: Centralize Financial Data Provide a single, unified platform where users can record, view, and manage all their financial transactions — eliminating the need for spreadsheets, notebooks, or multiple disconnected apps.

Objective 2: Deliver Real-Time Budget Intelligence Automatically calculate budget utilization in real-time as transactions are added, eliminating the manual effort of tracking budgets and providing instant visual feedback on spending health.

Objective 3: Democratize Financial Advice Make high-quality, personalized financial coaching accessible to everyone — not just those who can afford a human financial advisor — by leveraging the power of state-of-the-art AI language models.

Objective 4: Encourage Positive Financial Habits Use gamification (points, levels, badges) to motivate users to consistently track their finances, creating a positive feedback loop that builds long-term financial discipline.

Objective 5: Ensure Data Security Implement enterprise-grade security practices so users can trust the platform with their sensitive financial information, including complete data isolation between user accounts.

Objective 6: Provide an Exceptional User Experience Design a UI/UX that is not only functional but genuinely enjoyable to use — setting a new standard for personal finance apps with its premium glassmorphism aesthetic and smooth animations.

2.4 System Constraints and Assumptions

Technical Constraints

Constraint 1: Browser-Based Only Thrifty is a web application and does not have native iOS or Android mobile apps. It is designed to be responsive and work well on mobile browsers, but a dedicated app experience requires a future React Native port.

Constraint 2: No Live Bank API Integration Direct integration with Indian bank APIs requires RBI licensing (as a Payment Aggregator or Account Aggregator). Thrifty relies on manual transaction entry and future SMS parsing as workarounds.

Constraint 3: AI Rate Limits The free tiers of Google Gemini and Anthropic Claude have API rate limits. Heavy usage by many users simultaneously may require upgrading to paid API tiers with higher quotas.

Constraint 4: Storage Limitations Receipt image storage is dependent on the server's file storage. In the free deployment tier (Railway), storage is ephemeral, meaning uploaded receipts may not persist across deployments.

Assumptions

- Users have access to a modern web browser (Chrome, Firefox, Safari, Edge).
- Users have an internet connection for accessing the web app and AI features.
- Users are willing to manually enter their transactions (at least initially).
- The deployment environment (Railway/Vercel) provides sufficient uptime for production use.

2.5 System Benefits

Benefits to Individual Users

Financial Clarity: Users gain a clear, real-time picture of their financial health — income vs. spending — which is the first step toward making better financial decisions.

Proactive Alerts: Rather than discovering overspending at month-end, users receive real-time alerts when approaching budget limits, enabling course correction.

AI Coaching: Personalized AI advice adapts to each user's unique financial situation, providing actionable steps rather than generic platitudes.

Motivation: The gamification system creates a reward loop that encourages daily use, helping users build the habit of financial tracking.

Time Savings: Automated calculations, instant budget sync, and AI-generated reports save users hours of manual spreadsheet work each month.

Benefits to Society

Financial Literacy: By making financial management tools accessible and engaging, Thrifty contributes to improving financial literacy across demographics.

Economic Empowerment: Helping individuals manage money better leads to reduced debt, increased savings, and improved economic resilience.

Stress Reduction: Financial stress is a leading cause of mental health issues. By giving users control over their finances, Thrifty contributes to improved wellbeing.

TOPIC 3: TECHNOLOGY STACK

3.1 Frontend Technologies

React 18 and Vite

The Thrifty frontend is built using **React 18**, the latest major version of the React JavaScript library developed by Meta. React's component-based architecture makes it ideal for building complex, interactive user interfaces with reusable building blocks.

Why React?

- **Component Reusability**: UI elements like cards, modals, and charts are built once and reused throughout the application.
- **Virtual DOM**: React's Virtual DOM diffing algorithm ensures only the changed parts of the UI re-render, delivering excellent performance.
- **Ecosystem**: Massive ecosystem of libraries (Framer Motion, Recharts, Lucide Icons) accelerates development.

- **Hooks**: React Hooks (useState, useEffect, useContext) enable clean, functional component patterns without class complexity.

Vite as the Build Tool: Vite (French for "fast") replaces traditional Create React App (CRA) as the build tool. Vite uses native ES modules during development for near-instant Hot Module Replacement (HMR), making the developer experience significantly faster.

Feature	Create React App	Vite
Dev Server Start	~30s	~300ms
HMR Speed	~2s	~50ms
Bundle Size	Larger	Smaller
Modern ESM	❌	☑

Framer Motion

Framer Motion is the animation library used to create smooth, physics-based animations throughout the Thrifty UI. It provides:

- **Page Transitions**: Smooth fade and slide animations when navigating between pages.
- **Component Animations**: Cards and modals animate in with spring physics for a premium feel.
- **Gesture Animations**: Hover effects and press animations on interactive elements.
- **AnimatePresence**: Handles exit animations for components that unmount (e.g., modals closing).

Recharts

Recharts is a composable charting library built on React and D3.js. It powers all the data visualizations in the Thrifty dashboard:

- **PieChart/DonutChart**: Category-wise spending breakdown.
- **BarChart**: Monthly income vs. expense comparison.
- **AreaChart**: Spending trends over time.
- **Responsive Container**: Charts automatically resize to fit their parent container.

Lucide Icons

Lucide Icons provides a comprehensive set of clean, consistent SVG icons used throughout the UI. Unlike Font Awesome (which loads a large font file), Lucide ships individual SVG components, keeping the bundle size minimal.

Firebase Authentication

Firebase Authentication handles the Google OAuth 2.0 social login flow. It provides:

- Secure token management for Google Sign-In.
- Cross-platform authentication support.
- Automatic token refresh.
- The Firebase-generated ID token is exchanged with the Django backend for a custom JWT.

3.2 Backend Technologies

Django 4.x

Django is the backend web framework used to build the Thrifty API. Django follows the "batteries included" philosophy, providing:

- **ORM (Object-Relational Mapper)**: Defines database models in Python, abstracting SQL complexity.
- **Admin Interface**: Auto-generated admin panel for database management.
- **Migrations**: Version-controlled database schema changes.
- **Middleware**: JWT authentication, CORS handling, and security headers are applied via middleware.

Django's URL Routing: Django uses a URL dispatcher that maps URL patterns to Python view functions/classes, creating a clean separation between URL definition and business logic.

Django REST Framework (DRF)

Django REST Framework extends Django to build RESTful APIs. Key DRF features used in Thrifty:

- **ModelViewSets**: Automatically generates CRUD endpoints for each model.
- **Serializers**: Convert Python objects to JSON and validate incoming data.
- **Authentication Classes**: JWT Bearer token validation.
- **Permissions**: `IsAuthenticated` permission ensures only logged-in users access protected endpoints.
- **Pagination**: `PageNumberPagination` limits API response sizes.

Simple JWT

django-rest-framework-simplejwt provides JWT (JSON Web Token) authentication. When a user logs in, they receive an `access` token (short-lived, 5 minutes) and a `refresh` token (long-lived, 7 days). The frontend automatically refreshes the access token using the refresh token before it expires.

PostgreSQL (Production) / SQLite (Development)

SQLite is used in local development for zero-configuration simplicity. **PostgreSQL** is used in production (hosted on Railway) for:

- ACID compliance and data integrity.
- Advanced query capabilities.
- Concurrent multi-user read/write support.
- Production-grade reliability and scalability.

3.3 AI Integration Technologies

Google Gemini 1.5 Flash

Google Gemini 1.5 Flash is a state-of-the-art large language model (LLM) developed by Google DeepMind. It is optimized for:

- **Speed**: Significantly faster than Gemini Ultra, making it ideal for real-time chat applications.
- **Cost-Efficiency**: Lower API cost per token compared to larger models.

- **Long Context**: Supports up to 1 million token context window, allowing rich prompts with full financial context.
- **Multimodality**: Can process text (and optionally images like receipts) in future versions.

Integration in Thrifty: The `google-generativeai` Python SDK is used in the Django backend. When a user requests AI advice, the backend constructs a detailed system prompt containing the user's financial statistics (total income, total expenses, savings rate, top spending categories, active budgets) before sending the user's question to the Gemini API.

Anthropic Claude 3.5 Sonnet

Anthropic Claude 3.5 Sonnet is an advanced LLM known for:

- **Deep Reasoning**: Superior at complex multi-step reasoning tasks.
- **Instruction Following**: Highly effective at following specific formatting and behavioral instructions.
- **Safety**: Anthropic's Constitutional AI training makes Claude's outputs more reliable and aligned.

Dual AI Strategy: Thrifty allows users to choose between Gemini and Claude, providing flexibility. This also acts as a failover — if one API is unavailable, the user can switch to the other.

3.4 DevOps and Infrastructure Technologies

Railway (Backend Hosting)

Railway is the cloud Platform-as-a-Service (PaaS) used to host the Django backend. Railway provides:

- **Automatic Deployments**: Connects to the GitHub repository and redeploys on every push to main.
- **PostgreSQL Plugin**: A managed PostgreSQL database provisioned with one click.
- **Environment Variables**: Secure storage for secrets (API keys, database URLs).
- **Free Tier**: Sufficient for development and low-traffic production use.

Vercel (Frontend Hosting)

Vercel is the cloud platform used to host the React frontend. Vercel provides:

- **Global CDN**: Static assets served from edge locations worldwide for fast load times.
- **Automatic HTTPS**: SSL/TLS certificates automatically provisioned and renewed.
- **Preview Deployments**: Every pull request gets its own preview URL for testing.
- **Zero Configuration**: Vite projects deploy with zero configuration changes.

GitHub

GitHub is used for version control, code collaboration, and CI/CD triggers. The repository contains all source code, configuration files, and documentation.

3.5 Development Tools and Libraries

ESLint

ESLint is the JavaScript linting tool used to enforce code quality and consistency. The configuration extends `eslint:recommended` and `react/recommended` rules, catching potential bugs and style issues before they reach production.

Python Virtual Environment

Python's venv module creates an isolated Python environment for the backend, ensuring dependency versions are consistent across development and production environments.

dotenv

python-dotenv (backend) and **Vite's built-in .env support** (frontend) are used to manage environment variables, keeping sensitive keys (API keys, database URLs, secret keys) out of the source code.

django-cors-headers

django-cors-headers handles Cross-Origin Resource Sharing (CORS) — the mechanism that allows the React frontend (served from Vercel) to make API requests to the Django backend (served from Railway) despite being on different domains. Without proper CORS configuration, all API calls from the frontend would be blocked by browsers' security policies.

TOPIC 4: SYSTEM ARCHITECTURE

4.1 Overall System Architecture

Architecture Pattern: Client-Server with REST API

Thrifty follows a **Client-Server architecture** using a **RESTful API** as the communication layer between the frontend and backend. This separation of concerns provides several key advantages:

- **Independent Deployment**: Frontend and backend can be deployed, scaled, and updated independently.
- **Technology Flexibility**: Any client (web browser, mobile app, CLI) can consume the same REST API.
- **Testability**: The API can be tested independently using tools like Postman or curl.
- **Security**: Sensitive business logic and database credentials remain on the server side.

Three-Tier Architecture

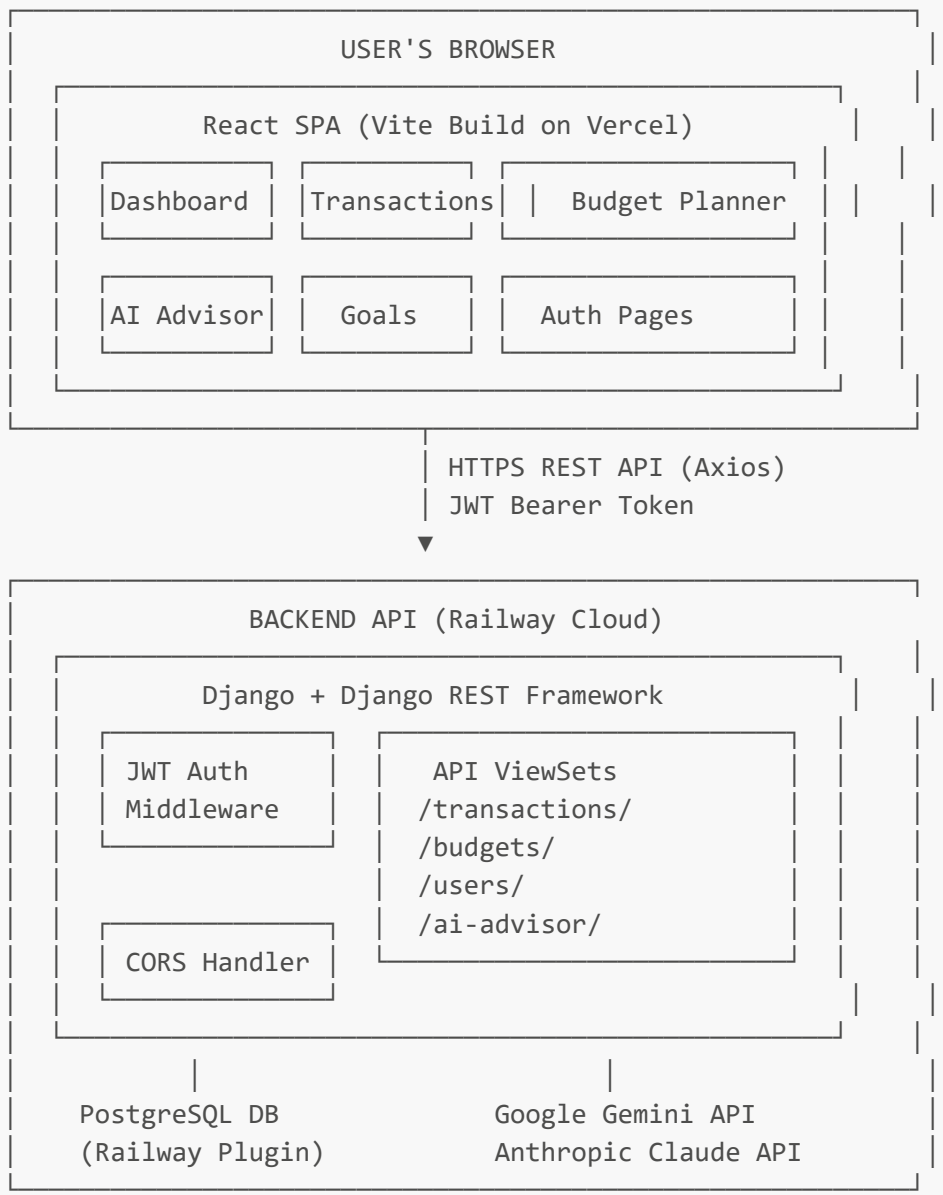
Thrifty implements a classic three-tier architecture:

Tier 1 – Presentation Layer (Frontend): The React SPA running in the user's browser. Handles UI rendering, user interaction, state management (React Context API), and communication with the backend via Axios HTTP calls.

Tier 2 – Application Layer (Backend): The Django REST API running on Railway. Handles business logic, data validation, authentication, authorization, and database operations. Also manages AI API calls on behalf of the frontend.

Tier 3 – Data Layer (Database): PostgreSQL on Railway (production) / SQLite (development). Stores all persistent data: user accounts, transactions, budgets, goals, badges, and user profiles.

System Diagram



4.2 Frontend Architecture

Component Hierarchy

The React frontend is organized into the following component hierarchy:

```
App.jsx (Root)
├── Context Providers
│   ├── AppContext (Global State)
│   └── AuthContext (User Auth State)
├── Router (React Router v6)
```

```
│   ├── PublicRoutes
│   │   ├── LoginPage
│   │   ├── RegisterPage
│   │   └── ForgotPasswordPage
│   └── ProtectedRoutes (requires authentication)
│       ├── Layout (Sidebar + Navbar)
│       │   ├── Dashboard
│       │   ├── TransactionList
│       │   ├── BudgetPlanner
│       │   ├── AIAdvisor
│       │   ├── Goals
│       │   └── Profile
│       └── NotFoundPage
```

State Management

Thrifty uses **React Context API** for global state management (instead of Redux, which is overkill for this scale):

- **AppContext**: Stores and provides global application data — transactions list, budgets list, loading states, and shared utility functions like `formatCurrency()`.
- **AuthContext**: Stores user authentication state — current user object, JWT tokens, and auth-related functions (login, logout, refreshToken).

API Communication (Axios)

All HTTP requests to the backend use **Axios**, configured with:

- **Base URL**: Points to the Railway backend URL.
- **Request Interceptor**: Automatically attaches the JWT `Authorization: Bearer <token>` header to every request.
- **Response Interceptor**: Catches 401 Unauthorized responses and automatically triggers a token refresh before retrying the original request.

4.3 Backend Architecture

Django App Structure

The Django backend is organized into focused apps:

```
backend/
├── manage.py
├── thrifty_backend/         # Project settings
│   ├── settings.py
│   ├── urls.py              # Root URL configuration
│   └── wsgi.py
├── users/                   # User management app
│   ├── models.py            # UserProfile, Badge models
│   ├── views.py             # Registration, Login, Google Auth views
│   ├── serializers.py
```

```
│   └── urls.py
├── transactions/           # Transaction management app
│   ├── models.py           # Transaction model
│   ├── views.py            # TransactionViewSet
│   ├── serializers.py
│   └── urls.py
├── budgets/                # Budget management app
│   ├── models.py           # Budget model
│   ├── views.py            # BudgetViewSet
│   ├── serializers.py
│   └── urls.py
└── ai_advisor/             # AI integration app
    ├── views.py            # Gemini & Claude API calls
    └── urls.py
```

Request Processing Pipeline

Every API request goes through the following pipeline:

1. **CORS Middleware** — Validates the request origin against the allowed domains whitelist.
2. **JWT Authentication Middleware** — Extracts and validates the Bearer token from the `Authorization` header.
3. **URL Router** — Maps the request URL to the appropriate ViewSet.
4. **Permission Check** — Verifies the user has the `IsAuthenticated` permission.
5. **ViewSet Logic** — Executes CRUD operations with automatic `user=request.user` filtering.
6. **Serializer** — Converts the database result to JSON.
7. **Response** — Returns the JSON response with appropriate HTTP status code.

4.4 Data Flow Architecture

Transaction Creation Flow

The following illustrates the complete data flow when a user creates a new transaction:

```
User fills form (React)
        │
        ▼
AppContext.addTransaction()
        │
        ▼
Axios POST /api/transactions/
  Headers: { Authorization: Bearer <jwt> }
  Body: { type, amount, category, date, description }
        │
        ▼
Django CORS Middleware ──► Validates origin
        │
        ▼
JWT Middleware ──────────► Decodes token, sets request.user
```

```
        |
        ▼
TransactionViewSet.create()
  - Validates serializer data
  - Sets transaction.user = request.user
  - Saves to PostgreSQL
  - Triggers post_save signals
        |
        ▼
Post-Save Signals:
  - Update UserProfile.points (+10)
  - Check Budget threshold
  - Check Goal progress
        |
        ▼
Return: HTTP 201 Created + transaction JSON
        |
        ▼
Axios Response in React
  - AppContext updates local transactions state
  - Dashboard auto-refreshes summary cards
  - Budget progress bars re-calculate
```

4.5 Security Architecture

Multi-Layer Security

Thrifty implements a multi-layer security model:

Layer 1 – Transport Security (HTTPS) All communication between the browser and servers is encrypted via TLS/HTTPS. Both Railway and Vercel automatically provision and renew SSL certificates.

Layer 2 – Authentication (JWT) JSON Web Tokens are signed with a private `SECRET_KEY`. Any tampered token will fail signature verification and be rejected. Access tokens expire in 5 minutes; refresh tokens expire in 7 days.

Layer 3 – Authorization (Row-Level Security) Every database query is automatically filtered by `user=request.user`. Even if an authenticated user guesses a transaction ID belonging to another user, the query returns 404 (not found) because the filter excludes it.

Layer 4 – CORS Whitelist Only requests from pre-approved origins (the Vercel frontend URL) are accepted by the Django backend. This prevents cross-site request forgery from unauthorized domains.

Layer 5 – Environment Variables All secrets (database URL, API keys, Django SECRET_KEY) are stored as environment variables, never committed to the source code repository.

TOPIC 5: USER AUTHENTICATION & SECURITY

5.1 Authentication Overview

What is Authentication?

Authentication is the process of verifying the identity of a user attempting to access a system. In Thrifty, authentication ensures that:

1. Only registered users can access the application.
2. Each user's data remains completely private and separate from other users.
3. Sessions are managed securely with automatic token expiry and refresh.

Authentication Methods in Thrifty

Thrifty supports two authentication methods:

Method 1: Email/Password Authentication Traditional credential-based login where users register with their email address and a password. Passwords are never stored in plaintext — Django's `AbstractUser` model automatically hashes passwords using **PBKDF2 with SHA256** (a cryptographic hashing algorithm) before storing them.

Method 2: Google OAuth 2.0 (Social Login) Users can sign in with their Google account in a single click. This is implemented using **Firebase Authentication** on the frontend, which handles the OAuth 2.0 flow with Google's identity servers. The resulting Firebase ID token is sent to the Django backend, which verifies it with Google's public keys and creates/retrieves a corresponding user account.

5.2 JWT Token System

What is JWT?

JSON Web Token (JWT) is an open standard (RFC 7519) for securely transmitting information between parties as a compact, URL-safe JSON object. A JWT consists of three parts separated by dots:

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxfQ.abc123xyz
     HEADER                  PAYLOAD          SIGNATURE
```

Header: Contains the algorithm used for signing (typically HS256 — HMAC with SHA-256). **Payload**: Contains the "claims" — user data like `user_id`, `username`, and expiration time (`exp`). **Signature**: The header and payload encoded and signed with the server's `SECRET_KEY`. Any modification to the payload invalidates the signature.

Token Types

Access Token:

- Short-lived (5-minute expiry in Thrifty).
- Sent in the `Authorization: Bearer <token>` header with every API request.
- If stolen, the attacker can only misuse it for 5 minutes.

Refresh Token:

- Long-lived (7-day expiry in Thrifty).
- Stored securely in the browser's localStorage.
- Used to obtain a new access token when the current one expires.
- If stolen, revocation is possible by the server maintaining a token blacklist.

Token Flow Diagram

```
1. User logs in with credentials
        |
        ▼
2. Django validates credentials
        |
        ▼
3. Returns: { access: "eyJ...", refresh: "eyJ..." }
        |
        ▼
4. Frontend stores both tokens in localStorage
        |
        ▼
5. Every API request:
   Headers: { Authorization: Bearer <access_token> }
        |
        ▼
6. When access token expires (401 response):
   - Frontend intercepts 401
   - Sends refresh token to /api/token/refresh/
   - Receives new access token
   - Retries original request
```

5.3 Google OAuth 2.0 Integration

OAuth 2.0 Flow

Google OAuth 2.0 uses the **Authorization Code Flow** for web applications. The implementation in Thrifty works as follows:

Step 1: User clicks "Sign in with Google" button on the Thrifty login page. **Step 2:** Firebase Authentication SDK opens a Google sign-in popup. **Step 3:** User selects their Google account and grants permissions. **Step 4:** Google's servers issue a Firebase ID token to the frontend. **Step 5:** The frontend sends this Firebase ID token to the Django backend's `/api/users/google-login/` endpoint. **Step 6:** Django verifies the Firebase ID token using Google's public key certificates. **Step 7:** If valid, Django creates a new user record (if first time) or retrieves the existing one. **Step 8:** Django issues its own JWT access and refresh tokens and returns them to the frontend. **Step 9:** The frontend stores the JWT tokens and the user is logged in.

COOP/COEP Configuration

Cross-Origin Opener Policy (COOP) headers must be carefully configured to allow the Google sign-in popup to communicate with the parent window. The Thrifty configuration sets `Cross-Origin-Opener-Policy: same-origin-allow-popups` to permit this while maintaining security.

5.4 Password Security

Password Hashing

Django uses **PBKDF2 (Password-Based Key Derivation Function 2)** with SHA-256 to hash passwords before storing them. The stored password hash looks like:

```
pbkdf2_sha256$600000$salt$hashedvalue
```

- **600,000 iterations** make brute-force attacks computationally expensive.
- **Salt** is a random string added before hashing, making rainbow table attacks useless.

Forgot Password Flow

Users who forget their password can reset it via email:

Step 1: User enters email on the Forgot Password page. **Step 2:** Django generates a unique, time-limited password reset token. **Step 3:** An email is sent to the user with a reset link containing the token. **Step 4:** User clicks the link, which opens the Reset Password page. **Step 5:** User enters a new password; Django validates the token and updates the hash.

5.5 Data Isolation and Authorization

Row-Level Data Isolation

Every piece of user data in Thrifty is tagged with a `user` foreign key at the database level. Every API query automatically filters by the authenticated user. This is implemented in Django ViewSets as:

```
def get_queryset(self):
    return Transaction.objects.filter(user=self.request.user)
```

This means:

- **User A can never see User B's transactions**, even if they know the transaction IDs.
- **Attempting to access another user's resource returns HTTP 404** (not 403 Forbidden, to avoid revealing that the resource exists).
- **Multi-tenant safety**: Thousands of users can use Thrifty simultaneously with zero data leakage.

CORS Protection

Django's `corsheaders` middleware is configured with an explicit whitelist:

```
CORS_ALLOWED_ORIGINS = [
    "https://thrifty-app.vercel.app",
    "http://localhost:5173",
]
```

Any request from an unlisted origin is rejected with a CORS error, preventing unauthorized websites from making API calls on behalf of Thrifty users.

[END OF PART 1 — Topics 1 through 5] [Continue with THRIFTY_REPORT_PART2.md for Topics 6 through 10] [Continue with THRIFTY_REPORT_PART3.md for Topics 11 through 15]